

IP packet-level encrypted traffic classification using machine learning with a light weight feature engineering method

Pengcheng Luo, Jian Chu, Genke Yang^{*}

Ningbo Artificial Intelligence Institute, Shanghai Jiao Tong University, 315016 Ningbo, PR China
Department of Automation, Shanghai Jiao Tong University, 200240 Shanghai, PR China

ARTICLE INFO

Keywords:

Encrypted traffic classification
IP packet
Feature engineering
Machine learning
Cyber security

ABSTRACT

As the importance of personal data privacy increases, traffic encryption has become an important topic in network communication. In the field of network security and management, the development of encrypted traffic classification technology has drawn attention to deep learning methods. For raw encrypted traffic, deep learning models can realize end-to-end classification with high accuracy. However, deep learning methods do not explain which part of the encrypted traffic is critical to classification, and that will limit their application in some cyber security scenarios that demand high interpretability. The approach proposed in this paper features a novel feature engineering method named “BITization” to determine the encoding method of features and a sliding window technique to explore which bytes contribute the most to the classification. The accuracy of classical machine learning methods in encrypted traffic is improved by at least 14.1% through the proposed feature engineering approach. In the experiments, the enhanced methods achieve a 98.6% average accuracy and a 98.5% average F1-score on the ISCX-VPN-Service, Cross-Platform-IOS, Cross-Platform-Android, and USTC-TFC2016 datasets, from which we believe a state-of-the-art performance is achieved.

1. Introduction

Network traffic classification is a technique to identify traffic from different applications and network services, which plays a crucial role in network management and security. For example, in the field of network management, traffic classification can be applied for network quality assurance, anomaly detection, and pricing strategy [1]. In the field of network security, traffic classification is usually used to discover malicious traffic [2,3], which can be generated by webshell and its management tools, such as Behinder, AntSword, etc. In recent years, with the growing public awareness of privacy issues on the Internet [4], encryption of network traffic using technologies such as Tor, VPN, and TLS/SSL has become a standard practice in the Internet communication. However, how to deal with encrypted traffic has brought new challenges to traditional traffic classification techniques and has attracted the attention of academia and industry.

Specifically, there are still some deficiencies with the current traffic classification method.

- The natural complexity of the encryption mechanism: the encryption procedure relies on the cryptographic algorithm and

pseudo-random mechanism to encrypt the original data, which gives the encrypted data high security, and resistivity to crack.

- Limitations of the traditional Deep Packet Inspection (DPI) traffic classification: the basic principle of DPI technology is to capture the pattern of payload in packets, which is encrypted and thus makes DPI difficult to be applied to encrypted traffic [5].
- In previous research, the traffic characterization and the application identification are rarely considered as the task of encrypted traffic classification simultaneously. Taking both granularities of the task into account puts a higher demand on the classification model.

The wide application of deep learning (DL) models in the industry is limited due to their complex architecture, high computational cost, and poor interpretability, despite the rapid development of DL technology. Classical Machine Learning (ML) is still the dominant technology in traditional industries in terms of practical application. By designing expert-features, ML models can achieve comparable accuracy to deep learning, while consuming significantly less computational resource. As excellent gradient boosting decision tree (GBDT) techniques [6], such

^{*} Corresponding author at: Ningbo Artificial Intelligence Institute, Shanghai Jiao Tong University, 315016 Ningbo, PR China.

E-mail address: gkyang@sjtu.edu.cn (G. Yang).

as XGBOOST [7] and LightGBM [8] are being developed, a growing number of researchers are applying these techniques to solve problems with industrial-scale massive data. For example, LightGBM is commonly used for multi-classification, click-through prediction, and search ranking tasks. As an implementation framework of the GBDT algorithm, LightGBM has the advantages of efficient training, low memory consumption, and less overfitting of the model [9].

In our model, raw encrypted traffic is input as feature into ML models to classify traffic in two granularities: traffic characterization and application identification. The main contributions of this paper can be summarized as follows:

- An innovative concept of “Optimal sliding window size”, providing guidance on constructing input features to optimize the classification performance on encrypted traffic;
- An unprecedented feature transformation method “BITization” to improve the accuracy of classical classifiers, such as Support Vector Machine (SVM), Softmax, Naive Bayesian (NB), and K-NearestNeighbor (KNN), by 23.7% on average;
- Conduct empirical studies on public encrypted traffic datasets, such as ISCX-VPN-Service, Cross-Platform, etc., to verify the superiority of the proposed method. The results show that our method achieves the optimal F1-score (F1) and optimal accuracy in three tasks out of five.

The remainder of this paper is organized as follows. In Section 2, a literature review of important methods in encrypted traffic classification is given. Section 3 introduces the proposed preprocessing methods, feature engineering methods, and the classification model. Section 4 describes experimental and evaluation results based on several public encrypted traffic datasets, where results and potential implications are also discussed. Finally, we conclude the paper in Section 5.

2. Related works

2.1. Traditional traffic classification

There are two typical types of traditional traffic classification approaches: port-based approach and payload inspection techniques. The port-based approach uses the port number information in the transport layer header as an important attribute for classification, and the port is also assumed to be associated with a specific application. However, this approach is proven to have poor performance in the circumstances of dynamic ports [10,11]. Payload inspection techniques are generally based on analyzing the payload of the application layer of the packet [12]. One of the well-known payload inspection techniques is DPI. DPI matches different protocols by setting predefined patterns and regular expressions as signatures [13,14]. However, this method is mostly ineffective in the case of new protocols, which requires adapted and updated corresponding patterns, as well as in handling encrypted traffic.

2.2. Statistical feature methods

Taylor et al. [15,16] constructed statistical features of network traffic to train SVM and Random Forest (RF) for application (app) classification. The model's features included the mean, variance, maxima, minima, and quantile of packet sizes in incoming and outgoing Transmission Control Protocol (TCP) streams respectively. The accuracy of the model on the top 100+ most popular apps in the app store is 99%, but its prediction results are highly dependent on the confidence of the model. Therefore, the classification accuracy is only 72% among all actual TCP streams. Al-Naami et al. [17] used statistical features of temporality to capture app behavior and an average accuracy of 92.6% is achieved in their paper. In methods based on statistical features, the performance of the method is more dependent on the expert-designed features. This can be attributed to the fact that the design of the features can significantly affect the generalization ability of the model.

2.3. Deep learning methods

Deep learning methods directly use raw encrypted traffic as input to achieve end-to-end traffic classification, and can avoid feature extraction process. Wang proposed a Stacked Auto-Encoder (SAE) method to classify Hyper Text Transfer Protocol (HTTP), Simple Mail Transfer Protocol (SMTP), and other protocol families in network traffic [18]. Sirinam et al. [19] used an architecture-specific Convolutional Neural Network (CNN) to construct the Deep-Fingerprinting model, which achieves 98% accuracy in Tor traffic using incoming and outgoing packet flow as input. Liu et al. [20] proposed an end-to-end Recurrent Neural Network (RNN) traffic classification model, which is called FS-NET. FS-NET takes packet length sequences as input and uses a bidirectional Gate Recurrent Unit (GRU) [21] model to learn traffic feature representation. It achieves a 99.14% true positive rate on a real-world dataset containing 18 applications. Lotfollahi et al. [22] constructed DeepPacket using SAE and 1D-CNN, where DeepPacket takes the IP packet of raw traffic as input. In their experiments, the ISCX-VPN-Service dataset is used and the results demonstrate that DeepPacket can achieve high accuracy of 98% and 94% on traffic characterization and application identification tasks, respectively. Lin et al. [23] used the Transformer model [24] to learn feature representations of the flow and packet of the raw encrypted traffic. Their model is trained with unsupervised pre-train and fine-tuning techniques [25]. The proposed ET-BERT has an F1 of 98.9% on the ISCX-VPN-Service dataset, which has a 5.2% improvement compared to other methods. Akbari et al. [26] designed features with good generalization, combined with Stacked Long Short-Term Memory (LSTM) layers and CNN for learning cryptographic protocols. For application classification task, it achieves 99% accuracy on QUIC dataset. Yang et al. [27] innovatively combined supervised learning with unsupervised techniques to design a CNN network for unknown traffic detection. Their method achieves over 90% accuracy on the top-200 application classification dataset.

Although DL methods have high accuracy in encrypted traffic classification, they are difficult to be widely used in production systems of traditional industries because of their complex models and high overhead costs [28]. For example, in Security Incident and Event Manager (SIEM) system of global banks, the amount of data grows massively. The deployment requirements of DL models under such conditions cannot be matched by the existing server capacity [29]. The classical ML approach is a better choice for non-profit sectors such as network and security departments, because ML models can be deployed without DL-dedicated servers, and the requirement for computational resources is much smaller.

3. Methods

After acquiring the encrypted traffic, IP packets in the traffic data will be transformed by BITization and optimal sliding window methods to create input features. The feature samples will then be input to the ML algorithm to learn an effective encrypted traffic classification model.

3.1. Packets preprocessing

IP packet p , which is a byte array, is given as Eq. (1), where n is the total bytes of IP packets.

$$p = (p_1, p_2, \dots, p_n) \quad (1)$$

In the data preprocessing stage, each IP packet in the traffic is processed to create features of the ML models. The process of raw traffic preprocessing is as follows:

Removal of IP address and port information: Although IP addresses and ports sometimes have a strong correlation with applications, they are removed according to previous research practice [22, 23]. Removal operation is defined as:

$$\text{RemoveIPAndPort}(p) = \text{slice}(p, 1, 12) + \text{slice}(p, 25, n) \quad (2)$$

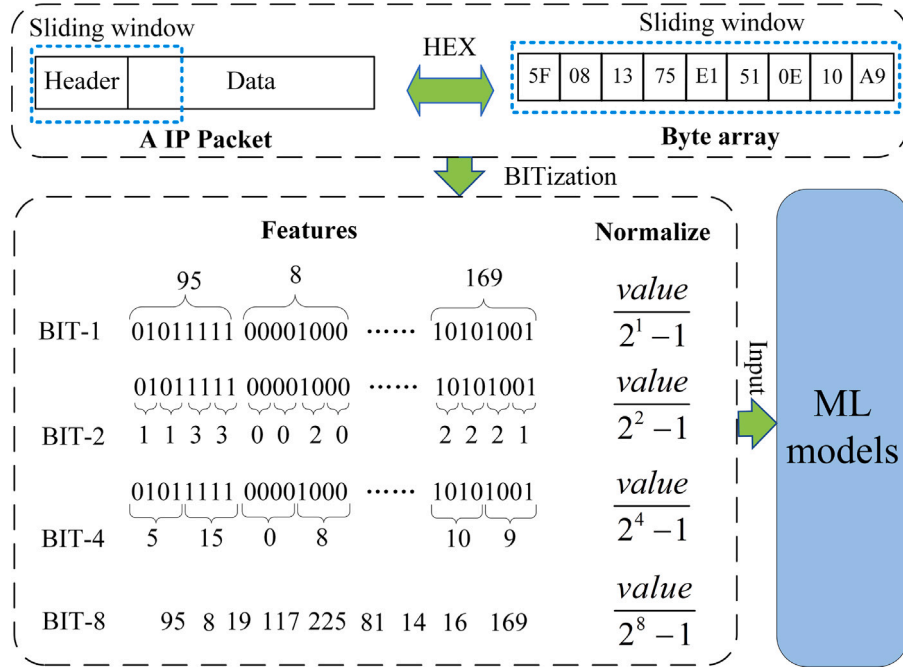


Fig. 1. Bitization features.

$\text{slice}(p, m, n)$ is a slice that contains the m th to the n th element from the array p (the m th and n th elements are included). $+$ means array concatenation. $\text{RemoveIPAndPort}(p)$ is the array after the removal operation.

UDP-Padding: The transport layer header is included in the IP packet. There are two main transport layer protocols: Transmission Control Protocol and User Datagram Protocol (UDP), which have different header lengths. TCP header is 20 bytes (excluding options), while UDP header occupies 8 bytes. According to Lotfollahi's study [22], data alignment of TCP header and UDP header can be achieved by padding operation. Specifically, 12 bytes are padded at the end of the UDP header with '0 × 00' (each occupying one byte) to make it the same length as the TCP header, as shown in Eq. (3).

$$\text{UDPPadding}(p) = \text{slice}(p, 1, 28) + '0x00' \times 12 + \text{slice}(p, 29, n) \quad (3)$$

$\text{UDPPadding}(p)$ is the array after UPD-Padding operation.

In the process of building the training set, the data cleaning procedure is performed first, specifically:

- **Outliers eliminating:** The majority of IP packets in the dataset belong to IPv4 protocol, and only 5%–10% of packets belong to IPv6 protocol or malformed packets, which is regarded as outliers in our experiment. The process of eliminating outliers is shown in Eq. (4).

$$\text{Filter}(p) = \begin{cases} 1 & \text{if } p_1 = '0x45' \\ 0 & \text{if } p_1 \neq '0x45' \end{cases} \quad (4)$$

In the IP header, '4' represents IPv4 and '5' represents the IP header length of 20 bytes. $\text{Filter}(p) = 0$ means that the packet p will be removed from the dataset.

- **Dataset balancing:** Since the disparity in the number of IP packets with different labels is overwhelming, ML models tend to categorize them into the class with more instances. Therefore, the undersampling technique is used to ensure that the number of IP packages with different labels is equal.

3.2. Feature engineering

3.2.1. Optimal sliding window

The maximum length of an IP packet is determined by the Maximum Transmission Unit (MTU), which is usually between 20 and 1500 bytes.

Considering that only some of the bytes in an IP packet may be useful for classification, a sliding window is used to search for key features, which are the input to the model.

For sliding windows, the window start position and window size are necessary parameters. According to a recent study [23], the information in the front of the IP packet can achieve higher classification accuracy than the information in the tail when used as input features. Therefore, the position of the sliding window is fixed at the head of the IP packet, and the sliding window size is adjusted for optimizing model performance. The optimization problem can be defined as:

$$\begin{aligned} N^* &= \arg\max_N L(N) \\ \text{s.t. } &\begin{cases} N \in \{1, 2, \dots, 1500\} \\ w_d = 1 \end{cases} \end{aligned} \quad (5)$$

N is the sliding window size, w_d is the window start position, $L(\bullet)$ is model performance function. Once the sliding window size N is obtained, the sliding window operation can be defined as:

$$\text{SlidingWindow}(p) = \text{slice}(p, 1, N) \quad (6)$$

$\text{SlidingWindow}(p)$ is the array after applying sliding window.

3.2.2. BITization

To build the input features, a light weight feature transformation method named "BITization" by converting the byte array p to re-sampled array x is proposed. BITization contains four different feature transformation methods: BIT-1, BIT-2, BIT-4, and BIT-8. Specifically, BITization converts each element of the byte array to binary. After that, they are re-encoded to integers and combined in the original order of arrangement, as shown in Fig. 1.

The process of BITization can be defined as follows,

$$x = \text{BIT}(p, k) \quad (7)$$

x is the BITization feature, k is the input option, and $n \in \{1, 2, 4, 8\}$. BIT-1 is encoding the input in binary form, and BIT-8 is encoding the input in decimal form. BIT-2 and BIT-4 can be considered as intermediate states between BIT-1 and BIT-8.

Table 1
The statistical information of datasets.

Task	Abbre.	Dataset	Label	Packet
ISCX-VPN-App	APP	ISCX-VPN-Service	16	40.9 k
ISCX-VPN-Traffic	TRAFFIC	ISCX-VPN-Service	11	126.9 k
USTC-TFC2016	USTC	USTC-TFC2016	19	114 k
Cross-Platform-IOS	IOS	Cross-Platform-IOS	185	185 k
Cross-Platform-Android	ANDROID	Cross-Platform-Android	167	167 k

3.3. Encrypted traffic classification and evaluation criteria

Single IP packets are input to the model to predict the class after creating the feature sample. All model classifications can be uniformly defined as follows,

$$y = f(x_{0-1}) \quad (8)$$

where y is the prediction class, and f is the classification model. x_{0-1} are the input features, as defined in the equation below. BITization feature x is generally normalized by 0–1 min–max normalization.

$$x_{0-1} = \frac{x}{2^k - 1} \quad (9)$$

To evaluate the classification performance of the model, we use accuracy (AC), precision (PR), recall (RC), and F1 as evaluation criteria. The above performance metrics can be expressed as:

$$AC = \frac{TP + TN}{TP + TN + FP + FN} \quad (10)$$

$$PR = \frac{TP}{TP + FP} \quad (11)$$

$$RC = \frac{TP}{TP + FN} \quad (12)$$

$$F1 = \frac{2 \times PR \times RC}{PR + RC} \quad (13)$$

TP, FP, FN, and TN stand for true positive, false positive, false negative, and true negative samples, respectively.

4. Experiment

4.1. Experimental settings

In our experiments, we choose four popular datasets in the field of traffic classification as the object of empirical research (ISCX-VPN-Service, Cross-Platform-IOS, Cross-Platform-Android, and USTC-TFC2016) [23]. The application identification performance and traffic characterization ability of our method are verified in these datasets. The details of the datasets are detailed in the Table 1.

In this paper, Python is used as the experimental platform. Scikit-learn [30] and Pytorch [31] are used to implement the ML methods. The ML methods include KNN, SVM, NB, SOFTMAX, CART, RF, XG-BOOST, and LightGBM. The algorithm is set up as follows: $n_{neighbors}$ is set to 10 in KNN; SVM uses Radial Basis Function (RBF) kernel; base learner for RF is CART; initial learning rate in SOFTMAX is 0.01 and the optimizer of SOFTMAX is Adam [32]. The ratio of the number of training and test data is set to 2:1, which is accomplished by stratified sampling.

4.2. Evaluation of the relationship between the sliding window size and model accuracy

We use sliding windows to select information in IP packets as input features for different models to investigate the relationship between the sliding window size and classification performance. In experiments, CART, SOFTMAX, NB, and KNN are used as classification models in USTC, IOS, and ANDROID datasets respectively, as shown in Fig. 2.

The 3 sub-figures represent 3 different datasets. 4 classification models are run to observe the accuracy using each dataset. The size of IP packets in the above datasets is standardized to 1500 bytes by padding. The position of the sliding window is fixed at the head of the IP packet, while the sliding window size is varied to observe the influence of the feature-length on the classification performance. In the USTC, IOS, and ANDROID datasets, the sliding window size is selected as 12, 50, 100, 200, and 300, as shown in the sub-figure “USTC”, “IOS”, and “ANDROID”.

According to the results in the sub-figure “USTC”, the accuracy of the four different models improves as the sliding window size gets larger. However, as the sliding window size becomes larger than a certain value, the accuracy of the models gradually decreases. Therefore, there is an optimal length of input features, which is defined as “optimal sliding window size” in this paper. In the sub-figure “IOS”, it can be observed that optimal sliding window size appears between 50 to 100. Similar phenomena can be observed in the other two sub-figures.

Furthermore, the accuracy of some specific models does not necessarily decrease as the length of the input features grows, for example, CART and SOFTMAX in sub-figure “USTC” according to the results. This can be explained by the insensitivity of the model to input feature length. In this case, the sliding window size with which the accuracy reaches its highest value for the first time can be defined as “optimal sliding window size” because a shorter input length means less computational resource consumption.

4.3. Evaluation of impact of BITization on model accuracy

To evaluate the impact of different BITization methods on the classification performance, experiments on different BITization methods BIT-1, BIT-2, BIT-4, and BIT-8 with RF, CART, SVM, SOFTMAX, KNN, and NB as models, were conducted in all datasets, as shown in Fig. 3.

In all sub-figures of Fig. 3, the accuracy of SVM, NB, and SOFTMAX decreases as they shift from BIT-1 to BIT-8. KNN does not turn out as expected in sub-figures “TRAFFIC”, “IOS”, and “ANDROID” using BIT-2, which can be explained by the fact that the BIT-2 encoding method destroys the key features that KNN relies on.

In contrast, in the experiments on CART and RF models, there is no significant improvement in accuracy. For decision tree methods, one-hot encoding will cause sample imbalance in the process of selecting the optimal splitting features. The BITization method can be regarded as a special one-hot encoding, as a result, it has no significant effect on the decision tree methods.

In the comparison between BIT-1 and BIT-8, the average decrease in accuracy is 32.35% for KNN, 14.15% for SVM, 25.73% for NB, and 22.70% for SOFTMAX on all datasets, which demonstrates that the BIT-1 feature has an improvement on the ML methods.

4.4. Evaluation of algorithm performance

In the experiments, State of the Art (SOTA) methods are used to validate the effectiveness of our improvements to ML methods, as is shown in Tables 2 and 3. In ML methods, sliding window size N is set to 100; KNN, SOFTMAX, NB, and SVM models use BIT-1; decision tree methods CART, RF, XGBOOST, and LightGBM use BIT-8.

In ML methods, SVM has the best performance among the non-decision tree models, with an average accuracy of 94.77% and an average F1 of 94.47%. LightGBM has the best average performance level among all models, whose F1 is optimal on all datasets. The best model among SOTA methods is DeepPacket, whose average accuracy and F1 are 4.42%, 4.40% worse than SVM, and 8.22%, 8.39% worse than LightGBM. According to the experimental results, SVM and LightGBM are superior to SOTA methods in accuracy, precision, recall, and F1, which can verify the effectiveness of ML methods with the proposed feature engineering methods.

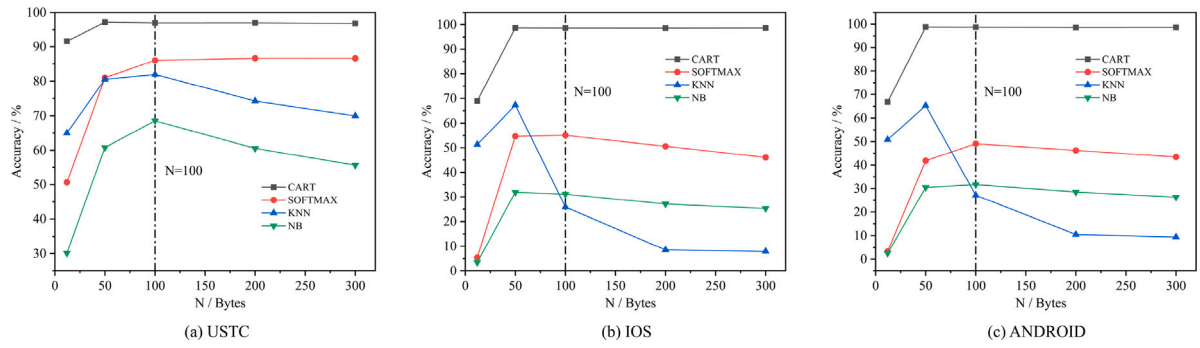


Fig. 2. Evaluation of the relationship between the sliding window size and model accuracy in different datasets.

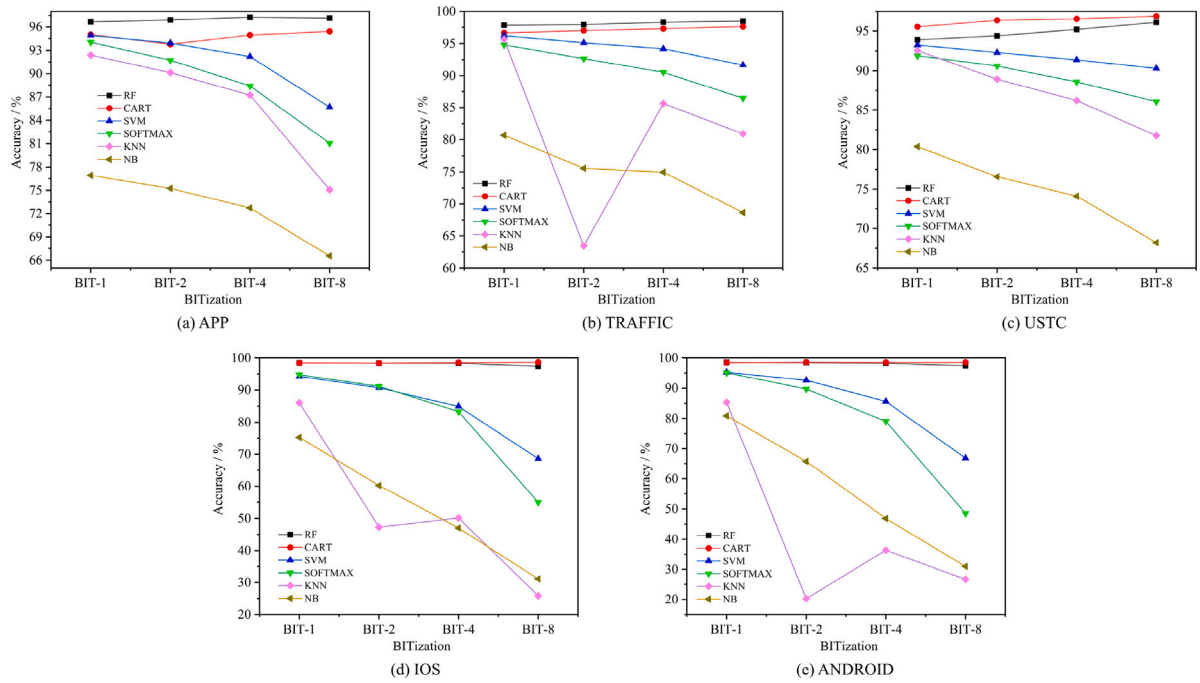


Fig. 3. Evaluation of the impact of different BITization features on classifier accuracy in all datasets.

Table 2

Results on APP, TRAFFIC and USTC.

Dataset	APP				TRAFFIC				USTC			
Metric (%)	AC	PR	RC	F1	AC	PR	RC	F1	AC	PR	RC	F1
ML methods with proposed feature engineering methods												
KNN	92.405	92.019	91.815	91.539	95.668	94.668	94.864	94.685	92.561	92.676	92.594	92.594
SVM	94.900	94.597	94.821	94.517	96.212	95.008	95.370	95.025	93.262	93.634	93.207	93.207
NB	76.930	74.487	75.965	73.849	80.671	78.679	78.975	78.323	80.369	85.656	79.800	79.800
SOFTMAX	94.050	92.937	92.871	92.875	94.801	92.973	93.189	93.030	91.907	91.850	91.786	91.786
CART	95.424	95.137	95.183	95.151	97.625	97.236	97.272	97.253	96.892	97.151	97.147	97.147
RF	97.130	96.939	97.072	96.988	98.485	97.954	98.176	98.058	96.106	96.440	96.147	96.147
XGBOOST	97.882	97.316	97.548	97.406	98.861	97.783	98.038	97.901	98.514	97.799	97.711	97.711
LightGBM	97.946	97.603	97.789	97.681	98.811	98.373	98.536	98.452	98.563	98.627	98.589	98.589
SOTA methods												
AppScanner [15]	69.933	70.156	67.828	66.858	88.230	88.255	87.727	87.559	64.279	81.198	64.378	66.810
CUMUL [33]	65.891	65.405	65.601	64.808	77.543	77.945	77.874	77.641	70.560	82.990	70.575	72.048
DeepFinger [19]	30.394	33.472	31.033	26.437	64.278	65.352	63.713	62.497	47.152	43.314	47.068	40.815
FS-NET [20]	67.849	69.174	67.756	67.564	80.736	81.469	81.412	81.379	69.826	84.834	70.400	71.278
DeepPacket [22]	89.239	88.472	88.640	88.451	94.721	94.136	94.315	94.209	90.755	91.033	90.755	90.769

Table 3
Results of IOS, ANDROID dataset.

Dataset	IOS				ANDROID				Average			
Metric (%)	AC	PR	RC	F1	AC	PR	RC	F1	AC	PR	RC	F1
ML methods with proposed feature engineering methods												
KNN	86.067	87.828	86.278	86.595	85.352	86.479	85.247	85.459	90.411	90.734	90.160	90.162
SVM	94.233	94.855	94.452	94.519	95.270	95.423	95.025	95.105	94.775	94.703	94.575	94.479
NB	75.286	83.098	75.661	77.618	80.907	86.154	80.809	82.315	78.833	81.615	78.242	78.383
SOFTMAX	94.781	93.287	93.184	93.211	95.091	93.503	93.407	93.432	94.126	92.910	92.887	92.865
CART	98.641	98.681	98.677	98.668	98.501	98.714	98.711	98.711	97.417	97.384	97.398	97.386
RF	97.396	97.451	97.427	97.420	97.393	97.450	97.391	97.396	97.302	97.247	97.243	97.202
XGBOOST	98.537	98.846	98.755	98.722	98.596	98.625	98.613	98.615	98.478	98.074	98.133	98.070
LightGBM	98.727	98.737	98.737	98.733	98.820	98.877	98.869	98.871	98.573	98.443	98.504	98.465
SOTA methods												
AppScanner [15]	32.515	36.124	32.477	33.022	48.365	53.964	48.017	49.416	60.664	65.939	60.085	60.733
CUMUL [33]	45.782	44.864	46.092	44.750	55.386	56.008	55.106	54.368	63.032	65.442	63.050	62.723
DeepFinger [19]	29.466	29.479	28.161	26.865	30.159	31.813	28.716	26.820	40.290	40.686	39.738	36.687
FS-NET [20]	46.614	46.483	47.098	45.886	56.456	59.535	56.943	57.056	64.296	73.753	64.722	64.633
DeepPacket [22]	92.972	93.019	92.973	92.940	84.061	84.188	84.061	83.994	90.350	90.170	90.149	90.073

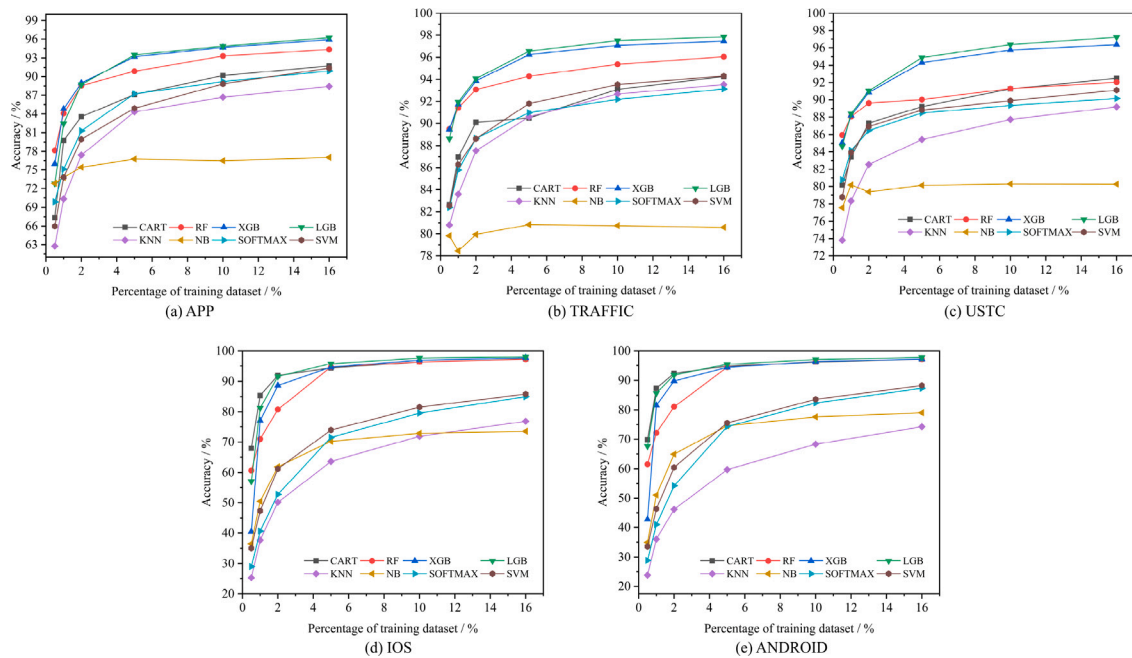


Fig. 4. Results of few-shot experiment.¹

4.5. Few-shot experiment

To evaluate the impact of the quantity of training data on model accuracy in the encrypted traffic classification problem, the trend of model accuracy is explored by adjusting the size of the training set. In our experiments, the ratio of the test data to the whole dataset is set to 1:3. The ratio of the training data to the whole dataset varies by stratified sampling: 0.5%, 1%, 2%, 5%, 10%, 16%. The performance of the models under different training data ratios in all datasets can be referred to Fig. 4.

In Fig. 4, the accuracy of the model increases as the training ratio increases. The models are less sensitive to the amount of training data on the APP, TRAFFIC, and USTC datasets compared to the IOS and ANDROID datasets. When the ratio changes from 0.5% to 16%, the average increase in accuracy on TRAFFIC, USTC, and APP datasets is 8.92%, 10.26%, and 20.02%. While in IOS and ANDROID datasets, it is

44.99% and 44.43%. Taking LightGBM as an example, when the ratio changes from 0.5% to 16%, the average increase on IOS and ANDROID datasets is 35.63%. However, the average increase is only 15.05% on the rest of the datasets. In conclusion, the performance of a model has different sensitivity to the quantity of training data on different datasets.

4.6. Visual analysis of classification

In the visual analysis part, T-distributed Stochastic Neighbor Embedding (T-SNE) is used to reduce the dimension of prediction probability, so as to intuitively explore the difference in classification performance between the proposed method and SOTA methods in a two-dimensional plane. DeepPacket, which performs best among SOTA methods, is used as the model for comparison. As shown in the legend of Fig. 5, different colors represent different sample labels. Although both DeepPacket and LightGBM can distinguish samples with different labels to some extent, DeepPacket appears to mix labels in some regions. For example, in the middle of Fig. 5(b), samples belonging to

¹ LGB refers to LightGBM, and XGB refers to XGBOOST.

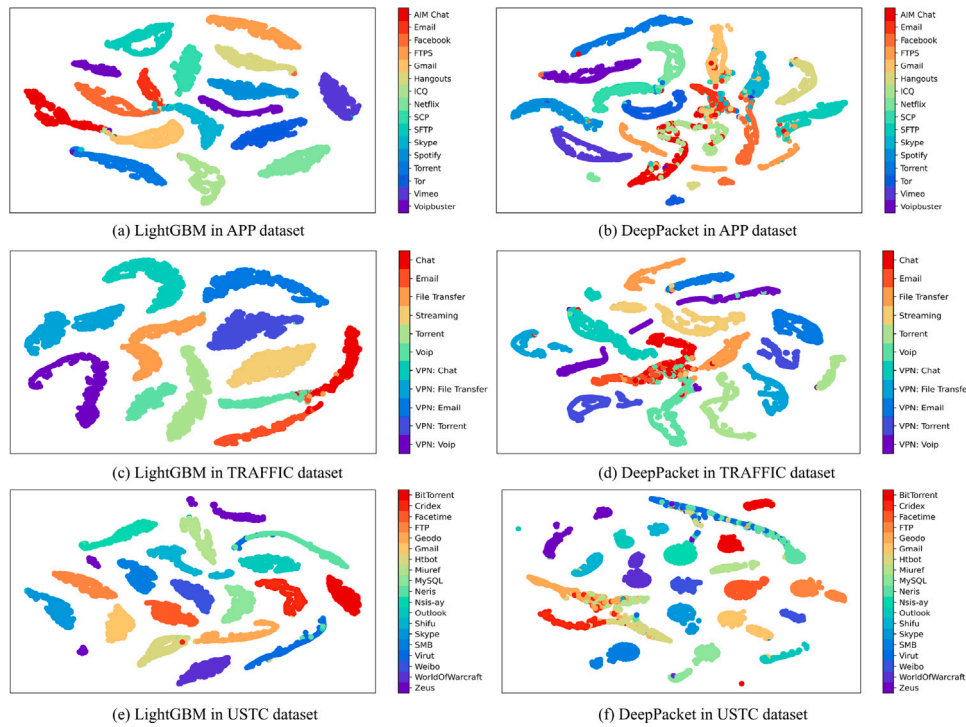


Fig. 5. Visualization results of LightGBM and DeepPacket.

Table 4
Results of experiments on the effect of the payload.

Accuracy(%)	LightGBM		KNN		SVM		SOFTMAX		Average		
	WITH ^a	NO	WITH	NO	WITH	NO	WITH	NO	WITH	NO	GAP ^b
APP	97.946	96.425	92.405	91.838	94.900	92.223	94.05	87.841	94.825	92.082	2.744
TRAFFIC	98.811	98.204	95.668	95.214	96.212	95.398	94.801	92.147	96.373	95.241	1.132
USTC	98.563	98.488	92.561	91.656	93.262	94.057	91.907	89.817	94.073	93.505	0.569
IOS	98.727	80.188	86.067	82.256	94.233	72.312	94.781	51.877	93.452	71.658	21.794
ANDROID	98.820	80.501	85.352	82.227	95.270	71.541	95.091	48.673	93.633	70.736	22.898

^aWITH corresponds to having the payload. NO corresponds to removing the payload.

^bGAP corresponds to the difference between two results.

Table 5
Results of additional experiments on the effect of the payload in CSTNET-TLS 1.3 dataset.

Accuracy(%)	CART		SVM		LightGBM		DeepPacket	
	WITH	NO	WITH	NO	WITH	NO	WITH	NO
CSTNET-TLS 1.3	80.645	84.844	33.761	44.497	81.132	78.558	66.293	64.414

“ICQ”, “AIM Chat”, and “Email” are not clearly separated. A similar phenomenon is observed in Fig. 5(d) and (f). Therefore, it is evident according to the visualization analytics that our proposed method has a stronger classification capability than the existing SOTA methods.

4.7. Evaluating the impact of payload on model accuracy

When the length of the sliding window $N \geq 50$, the feature is mainly composed of IP headers, transport layer headers, and a small amount of payload, i.e. data in IP packets. We conducted experiments to investigate whether the payload has an important impact on the model performance. In the experiments, the feature without payload is input to LightGBM, KNN, SVM, and SOFTMAX classifiers. The results are compared to the previous results for evaluation, as detailed in Table 4.

In APP, TRAFFIC, and USTC datasets, the average accuracy of the algorithms decreases by 2.74%, 1.13%, and 0.57% after the payload is completely removed from the feature. According to the results of experiments, the decrease in accuracy of IOS and ANDROID datasets is arranged from 3% to 47%. The degradation varies widely between

classifiers, for example, the accuracy of KNN and LightGBM decreases by 3.4% and 18.4% respectively, while SOFTMAX decreases by 44.6%.

For APP, TRAFFIC, and USTC datasets, the performance of classifiers is less affected by removing the payload, and they maintain a good performance. For IOS and ANDROID datasets, the accuracy of all classifiers decreases to various degrees when the payload is removed. The experimental results show that the model can learn useful information for classification from the payload, however, the impact of removing the payload on the performance varies according to the dataset and the classifier.

In addition, CSTNET-TLS 1.3 dataset [23] is used in our experiments. The dataset is based on the encryption protocol TLS 1.3, and includes a total of 511.99 K IP packets from 120 applications. The additional experimental results are shown in Table 5. CART and SVM have a higher accuracy without payload. This can be explained that the introduction of payload also brings noise to the features, which makes the performance of the learner degrade. However, for DeepPacket and LightGBM, the accuracy decreases after removing the payload, which can be explained by its dependency on the payload for classification.

From the additional experimental results, we can conclude that when the IP packets are encrypted by TLS 1.3, the impact of payload on the accuracy is within 12% for all classifiers. In particular, for classifiers with good classification performance, the impact can be controlled within 5%.

5. Conclusion

In this paper, a light weight feature engineering method is proposed, which can be well combined with current ML methods, achieving efficient classification of raw encrypted traffic. To be more specific, an innovative concept of “optimal sliding window size” is proposed first by investigating the relationship between the feature-length and classification accuracy. A light feature transformation technique “BITization” is also proposed, which can further improve the average classification accuracy of classical ML classifiers. In the experiment part, an empirical study of SOTA methods is accomplished, and it is concluded that LightGBM has the best performance in IP packet-level classification. Last, visual analysis and a few-shot experiment are designed to intuitively understand the classification performance with different models and evaluate the impact of train data in encrypted traffic classification. Given the implicit nature of malicious encrypted traffic, we will further extend our study to implement malicious traffic detection in the future.

CRedit authorship contribution statement

Pengcheng Luo: Conceptualization, Methodology, Software, Investigation, Formal analysis, Writing – original draft. **Jian Chu:** Supervision, Funding acquisition, Resources. **Genke Yang:** Supervision, Funding acquisition, Resources, Writing – review & editing.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

Data will be made available on request.

Acknowledgment

The research is supported by the National Key Research and Development Program of China (Grant No. 2020YFB1711200).

References

- [1] Dainotti A, Pescapé A, Claffy KC. Issues and future directions in traffic classification. *IEEE Netw* 2012;26(1):35–40.
- [2] Ahmad R, Alsmadi I, Alhamdani W, Tawalbeh L. A deep learning ensemble approach to detecting unknown network attacks. *J Inf Secur Appl* 2022;67:103196.
- [3] Liu J, Liu Y, Li J, Sun W, Cheng J, Zhang R, et al. Two statistical traffic features for certain APT group identification. *J Inf Secur Appl* 2022;67:103207.
- [4] Velan P, Čermák M, Čeleda P, Drašar M. A survey of methods for encrypted traffic classification and analysis. *Int J Netw Manage* 2015;25(5):355–74.
- [5] Wang Z, Fok KW, Thing VL. Machine learning for encrypted malicious traffic detection: Approaches, datasets and comparative study. *Comput Secur* 2022;113:102542.
- [6] Friedman JH. Greedy function approximation: a gradient boosting machine. *Ann Statist* 2001;1189–232.
- [7] Chen T, Guestrin C. Xgboost: A scalable tree boosting system. In: *Proceedings of the 22nd ACM SIGKDD international conference on knowledge discovery and data mining*. 2016, p. 785–94.
- [8] Ke G, Meng Q, Finley T, Wang T, Chen W, Ma W, et al. Lightgbm: A highly efficient gradient boosting decision tree. *Adv Neural Inf Process Syst* 2017;30.
- [9] Riera TS, Higuera J-RB, Higuera JB, Herraz J-JM, Montalvo J-AS. A new multi-label dataset for web attacks CAPEC classification using machine learning techniques. *Comput Secur* 2022;102788.
- [10] Constantinou F, Mavrommatis P. Identifying known and unknown peer-to-peer traffic. In: *Fifth IEEE international symposium on network computing and applications*. IEEE; 2006, p. 93–102.
- [11] Erman J, Mahanti A, Arlitt M, Williamson C. Identifying and discriminating between web and peer-to-peer traffic in the network core. In: *Proceedings of the 16th international conference on world wide web*. 2007, p. 883–92.
- [12] Khalife J, Hajjar A, Diaz-Verdejo J. A multilevel taxonomy and requirements for an optimal traffic-classification model. *Int J Netw Manage* 2014;24(2):101–20.
- [13] Sen S, Spatscheck O, Wang D. Accurate, scalable in-network identification of p2p traffic using application signatures. In: *Proceedings of the 13th international conference on world wide web*. 2004, p. 512–21.
- [14] Yeganeh SH, Eftekhari M, Ganjali Y, Keralapura R, Nucci A. Cute: Traffic classification using terms. In: *2012 21st international conference on computer communications and networks*. IEEE; 2012, p. 1–9.
- [15] Taylor VF, Spolaor R, Conti M, Martinovic I. Appscanner: Automatic fingerprinting of smartphone apps from encrypted network traffic. In: *2016 IEEE European symposium on security and privacy*. IEEE; 2016, p. 439–54.
- [16] Taylor VF, Spolaor R, Conti M, Martinovic I. Robust smartphone app identification via encrypted network traffic analysis. *IEEE Trans Inf Forensics Secur* 2017;13(1):63–78.
- [17] Al-Naami K, Chandra S, Mustafa A, Khan L, Lin Z, Hamlen K, et al. Adaptive encrypted traffic fingerprinting with bi-directional dependence. In: *Proceedings of the 32nd annual conference on computer security applications*. 2016, p. 177–88.
- [18] Wang Z. The applications of deep learning on traffic identification. *BlackHat USA* 2015;24(11):1–10.
- [19] Sirinam P, Imani M, Juarez M, Wright M. Deep fingerprinting: Undermining website fingerprinting defenses with deep learning. In: *Proceedings of the 2018 ACM SIGSAC conference on computer and communications security*. 2018, p. 1928–43.
- [20] Liu C, He L, Xiong G, Cao Z, Li Z. Fs-net: A flow sequence network for encrypted traffic classification. In: *IEEE INFOCOM 2019-IEEE conference on computer communications*. IEEE; 2019, p. 1171–9.
- [21] Cho K, Van Merriënboer B, Gulcehre C, Bahdanau D, Bougares F, Schwenk H, et al. Learning phrase representations using RNN encoder-decoder for statistical machine translation. 2014, arXiv preprint arXiv:1406.1078.
- [22] Lotfollahi M, Jafari Siavoshani M, Shirali Hossein Zade R, Saberian M. Deep packet: A novel approach for encrypted traffic classification using deep learning. *Soft Comput* 2020;24(3):1999–2012.
- [23] Lin X, Xiong G, Gou G, Li Z, Shi J, Yu J. ET-BERT: A contextualized datagram representation with pre-training transformers for encrypted traffic classification. In: *Proceedings of the ACM web conference 2022*. 2022, p. 633–42.
- [24] Vaswani A, Shazeer N, Parmar N, Uszkoreit J, Jones L, Gomez AN, et al. Attention is all you need. *Adv Neural Inf Process Syst* 2017;30.
- [25] Devlin J, Chang M-W, Lee K, Toutanova K. Bert: Pre-training of deep bidirectional transformers for language understanding. 2018, arXiv preprint arXiv:1810.04805.
- [26] Akbari I, Salahuddin MA, Ven L, Limam N, Boutaba R, Mathieu B, et al. A look behind the curtain: traffic classification in an increasingly encrypted web. *Proc ACM Meas Anal Comput Syst* 2021;5(1):1–26.
- [27] Yang L, Finamore A, Jun F, Rossi D. Deep learning and zero-day traffic classification: Lessons learned from a commercial-grade dataset. *IEEE Trans Netw Serv Manag* 2021;18(4):4103–18.
- [28] Tyagi AK, Rekha G. Challenges of applying deep learning in real-world applications. In: *Challenges and applications for implementing machine learning in computer vision*. IGI Global; 2020, p. 92–118.
- [29] González-Granadillo G, González-Zarzosa S, Diaz R. Security information and event management (SIEM): analysis, trends, and usage in critical infrastructures. *Sensors* 2021;21(14):4759.
- [30] Pedregosa F, Varoquaux G, Gramfort A, Michel V, Thirion B, Grisel O, et al. Scikit-learn: Machine learning in python. *J Mach Learn Res* 2011;12:2825–30.
- [31] Paszke A, Gross S, Massa F, Lerer A, Bradbury J, Chanan G, et al. Pytorch: An imperative style, high-performance deep learning library. *Adv Neural Inf Process Syst* 2019;32.
- [32] Kingma DP, Ba J. Adam: A method for stochastic optimization. 2014, arXiv preprint arXiv:1412.6980.
- [33] Panchenko A, Lanze F, Pennekamp J, Engel T, Zinnen A, Henze M, et al. Website fingerprinting at internet scale. In: *NDSS*. 2016.